JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class HijrahChronology

java.lang.Object
    java.time.chrono.AbstractChronology
        java.time.chrono.HijrahChronology

All Implemented Interfaces:

`Serializable, Comparable<Chronology>, Chronology`

```
public final class HijrahChronology
extends AbstractChronology
implements Serializable
```

The Hijrah calendar is a lunar calendar supporting Islamic calendars.

The HijrahChronology follows the rules of the Hijrah calendar system. The Hijrah calendar has several variants based on differences in when the new moon is determined to have occurred and where the observation is made. In some variants the length of each month is computed algorithmically from the astronomical data for the moon and earth and in others the length of the month is determined by an authorized sighting of the new moon. For the algorithmically based calendars the calendar can project into the future. For sighting based calendars only historical data from past sightings is available.

The length of each month is 29 or 30 days. Ordinary years have 354 days; leap years have 355 days.

CLDR and LDML identify variants:

Chronology ID	Calendar Type	Locale extension, see Locale	Description
Hijrah-umalqura	islamic-umalqura	ca-islamic-umalqura	Islamic - Umm Al-Qura calendar of Saudi Arabia

Additional variants may be available through `Chronology.getAvailableChronologies()`.

Example

Selecting the chronology from the locale uses `Chronology.ofLocale(Locale)` to find the Chronology based on Locale supported BCP 47 extension mechanism to request a specific calendar ("ca"). For example,

```
Locale locale = Locale.forLanguageTag("en-US-u-ca-islamic-umalqura");
Chronology chrono = Chronology.ofLocale(locale);
```

Implementation Requirements:

This class is immutable and thread-safe.

Implementation Note:

Each Hijrah variant is configured individually. Each variant is defined by a property resource that defines the `ID`, the `calendar type`, the start of the calendar, the alignment with the ISO calendar, and the length of each month for a range of years. The variants are loaded by HijrahChronology as a resource from hijrah-config-<calendar type>.properties.

The Hijrah property resource is a set of properties that describe the calendar. The syntax is defined by `java.util.Properties#load(Reader)`.

Property Name	Property value	Description
id	Chronology Id, for example, "Hijrah-umalqura"	The Id of the calendar in common usage
type	Calendar type, for example, "islamic-umalqura"	LDML defines the calendar types
version	Version, for example: "1.8.0_1"	The version of the Hijrah variant data
iso-start	ISO start date, formatted as `yyyy-MM-dd`, for example: "1900-04-30"	The ISO date of the first day of the minimum Hijrah year.
yyyy - a numeric 4 digit year, for example "1434"	The value is a sequence of 12 month lengths, for example: "29 30 29 30 29 30 30 30 29 30 29 29"	The lengths of the 12 months of the year separated by whitespace. A numeric year property must be present for every year without any gaps. The month lengths must be between 29-32 inclusive.

Additional variants may be added by providing configuration properties files in `<JAVA_HOME>/conf/chronology` directory. The properties files should follow the naming convention of `hijrah-config-<chronology id>_<calendar type>.properties`.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
`static final HijrahChronology`	**`INSTANCE`**	Singleton instance of the Islamic Umm Al-Qura calendar of Saudi Arabia.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`HijrahDate`	**`date`**`(int prolepticYear, int month, int dayOfMonth)`	Obtains a local date in Hijrah calendar system from the proleptic-year, month-of-year and day-of-month fields.
`HijrahDate`	**`date`**`(Era era, int yearOfEra, int month, int dayOfMonth)`	Obtains a local date in Hijrah calendar system from the era, year-of-era, month-of-year and day-of-month fields.
`HijrahDate`	**`date`**`(TemporalAccessor tempor`	Obtains a local date in this chronology from another temporal object.
`HijrahDate`	**`dateEpochDay`**`(long epochDay)`	Obtains a local date in the Hijrah calendar system from the epoch-day.
`HijrahDate`	**`dateNow`**`()`	Obtains the current local date in this chronology from the system clock in the default time-zone.
`HijrahDate`	**`dateNow`**`(Clock clock)`	

		Obtains the current local date in this chronology from the specified clock.
HijrahDate	**dateNow**(ZoneId zone)	Obtains the current local date in this chronology from the system clock in the specified time-zone.
HijrahDate	**dateYearDay** (int prolepticYear, int dayOfYear)	Obtains a local date in Hijrah calendar system from the proleptic-year and day-of-year fields.
HijrahDate	**dateYearDay**(Era era, int yearOfEra, int dayOfYear)	Obtains a local date in Hijrah calendar system from the era, year-of-era and day-of-year fields.
HijrahEra	**eraOf**(int eraValue)	Creates the HijrahEra object from the numeric value.
List<Era>	**eras**()	Gets the list of eras for the chronology.
String	**getCalendarType**()	Gets the calendar type of the Islamic calendar.
String	**getId**()	Gets the ID of the chronology.
boolean	**isLeapYear** (long prolepticYear)	Checks if the specified year is a leap year.
ChronoLocalDateTime <HijrahDate>	**localDateTime** (TemporalAccessor tempor	Obtains a local date-time in this chronology from another temporal object.
int	**prolepticYear**(Era era, int yearOfEra)	Calculates the proleptic-year given the era and year-of-era.
ValueRange	**range** (ChronoField field)	Gets the range of valid values for the specified field.

HijrahDate	**resolveDate**(Map<TemporalField, Long> fieldValues, ResolverStyle resolverSt	Resolves parsed ChronoField values into a date during parsing.
ChronoZonedDateTime<HijrahDate>	**zonedDateTime**(Instant instant, ZoneId zone)	Obtains a ChronoZonedDateTime in this chronology from an Instant.
ChronoZonedDateTime<HijrahDate>	**zonedDateTime**(TemporalAccessor tempor	Obtains a ChronoZonedDateTime in this chronology from another temporal object.

Methods declared in class AbstractChronology

compareTo, equals, hashCode, toString

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Chronology

epochSecond, epochSecond, getDisplayName, isIsoBased, period

Field Details

INSTANCE

```
public static final HijrahChronology INSTANCE
```

Singleton instance of the Islamic Umm Al-Qura calendar of Saudi Arabia. Other Hijrah chronology variants may be available from Chronology.getAvailableChronologies().

Method Details

getId

```
public String getId()
```

Gets the ID of the chronology.

The ID uniquely identifies the `Chronology`. It can be used to lookup the `Chronology` using `Chronology.of(String)`.

Specified by:

`getId` in interface `Chronology`

Returns:

the chronology ID, non-null

See Also:

`getCalendarType()`

getCalendarType

```
public String getCalendarType()
```

Gets the calendar type of the Islamic calendar.

The calendar type is an identifier defined by the *Unicode Locale Data Markup Language (LDML)* specification. It can be used to lookup the `Chronology` using `Chronology.of(String)`.

Specified by:

`getCalendarType` in interface `Chronology`

Returns:

the calendar system type; non-null if the calendar has a standard type, otherwise null

See Also:

`getId()`

date

```
public HijrahDate date(Era era,
                       int yearOfEra,
                       int month,
                       int dayOfMonth)
```

Obtains a local date in Hijrah calendar system from the era, year-of-era, month-of-year and day-of-month fields.

Specified by:

`date` in interface `Chronology`

Parameters:

`era` - the Hijrah era, not null

`yearOfEra` - the year-of-era

`month` - the month-of-year

dayOfMonth - the day-of-month

Returns:

the Hijrah local date, not null

Throws:

DateTimeException - if unable to create the date

ClassCastException - if the era is not a HijrahEra

date

```
public HijrahDate date(int prolepticYear,
                       int month,
                       int dayOfMonth)
```

Obtains a local date in Hijrah calendar system from the proleptic-year, month-of-year and day-of-month fields.

Specified by:

date in interface Chronology

Parameters:

prolepticYear - the proleptic-year

month - the month-of-year

dayOfMonth - the day-of-month

Returns:

the Hijrah local date, not null

Throws:

DateTimeException - if unable to create the date

dateYearDay

```
public HijrahDate dateYearDay(Era era,
                              int yearOfEra,
                              int dayOfYear)
```

Obtains a local date in Hijrah calendar system from the era, year-of-era and day-of-year fields.

Specified by:

dateYearDay in interface Chronology

Parameters:

era - the Hijrah era, not null

yearOfEra - the year-of-era

dayOfYear - the day-of-year

Returns:

the Hijrah local date, not null

Throws:

`DateTimeException` - if unable to create the date

`ClassCastException` - if the `era` is not a `HijrahEra`

dateYearDay

```
public HijrahDate dateYearDay(int prolepticYear,
                              int dayOfYear)
```

Obtains a local date in Hijrah calendar system from the proleptic-year and day-of-year fields.

Specified by:

`dateYearDay` in interface `Chronology`

Parameters:

`prolepticYear` - the proleptic-year

`dayOfYear` - the day-of-year

Returns:

the Hijrah local date, not null

Throws:

`DateTimeException` - if the value of the year is out of range, or if the day-of-year is invalid for the year

dateEpochDay

```
public HijrahDate dateEpochDay(long epochDay)
```

Obtains a local date in the Hijrah calendar system from the epoch-day.

Specified by:

`dateEpochDay` in interface `Chronology`

Parameters:

`epochDay` - the epoch day

Returns:

the Hijrah local date, not null

Throws:

`DateTimeException` - if unable to create the date

dateNow

```
public HijrahDate dateNow()
```

Description copied from interface: Chronology

Obtains the current local date in this chronology from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Specified by:

`dateNow` in interface `Chronology`

Returns:

the current local date using the system clock and default time-zone, not null

dateNow

```
public HijrahDate dateNow(ZoneId zone)
```

Description copied from interface: Chronology

Obtains the current local date in this chronology from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Specified by:

`dateNow` in interface `Chronology`

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current local date using the system clock, not null

dateNow

```
public HijrahDate dateNow(Clock clock)
```

Description copied from interface: Chronology

Obtains the current local date in this chronology from the specified clock.

This will query the specified clock to obtain the current date - today. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using dependency injection.

Specified by:

dateNow in interface Chronology

Parameters:

clock - the clock to use, not null

Returns:

the current local date, not null

date

```
public HijrahDate date(TemporalAccessor temporal)
```

Description copied from interface: Chronology

Obtains a local date in this chronology from another temporal object.

This obtains a date in this chronology based on the specified temporal. A TemporalAccessor represents an arbitrary set of date and time information, which this factory converts to an instance of ChronoLocalDate.

The conversion typically uses the EPOCH_DAY field, which is standardized across calendar systems.

This method matches the signature of the functional interface TemporalQuery allowing it to be used as a query via method reference, aChronology::date.

Specified by:

date in interface Chronology

Parameters:

temporal - the temporal object to convert, not null

Returns:

the local date in this chronology, not null

See Also:

ChronoLocalDate.from(TemporalAccessor)

localDateTime

```
public ChronoLocalDateTime<HijrahDate> localDateTime
(TemporalAccessor temporal)
```

Description copied from interface: Chronology

Obtains a local date-time in this chronology from another temporal object.

This obtains a date-time in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDateTime`.

The conversion extracts and combines the `ChronoLocalDate` and the `LocalTime` from the temporal object. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects. The result uses this chronology.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::localDateTime`.

Specified by:

`localDateTime` in interface `Chronology`

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local date-time in this chronology, not null

See Also:

`ChronoLocalDateTime.from(TemporalAccessor)`

zonedDateTime

```
public ChronoZonedDateTime<HijrahDate> zonedDateTime
(TemporalAccessor temporal)
```

Description copied from interface: `Chronology`

Obtains a `ChronoZonedDateTime` in this chronology from another temporal object.

This obtains a zoned date-time in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoZonedDateTime`.

The conversion will first obtain a `ZoneId` from the temporal object, falling back to a `ZoneOffset` if necessary. It will then try to obtain an `Instant`, falling back to a `ChronoLocalDateTime` if necessary. The result will be either the combination of `ZoneId` or `ZoneOffset` with `Instant` or `ChronoLocalDateTime`. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects. The result uses this chronology.

This method matches the signature of the functional interface TemporalQuery allowing it to be used as a query via method reference, aChronology::zonedDateTime.

Specified by:

zonedDateTime in interface Chronology

Parameters:

temporal - the temporal object to convert, not null

Returns:

the zoned date-time in this chronology, not null

See Also:

ChronoZonedDateTime.from(TemporalAccessor)

zonedDateTime

```
public ChronoZonedDateTime<HijrahDate> zonedDateTime(Instant instant,
                                                     ZoneId zone)
```

Description copied from interface: Chronology

Obtains a ChronoZonedDateTime in this chronology from an Instant.

This obtains a zoned date-time with the same instant as that specified.

Specified by:

zonedDateTime in interface Chronology

Parameters:

instant - the instant to create the date-time from, not null

zone - the time-zone, not null

Returns:

the zoned date-time, not null

isLeapYear

```
public boolean isLeapYear(long prolepticYear)
```

Description copied from interface: Chronology

Checks if the specified year is a leap year.

A leap-year is a year of a longer length than normal. The exact meaning is determined by the chronology according to the following constraints.

- a leap-year must imply a year-length longer than a non leap-year.
- a chronology that does not support the concept of a year must return false.

- the correct result must be returned for all years within the valid range of years for the chronology.

Outside the range of valid years an implementation is free to return either a best guess or false. An implementation must not throw an exception, even if the year is outside the range of valid years.

Specified by:

`isLeapYear` in interface `Chronology`

Parameters:

`prolepticYear` - the proleptic-year to check, not validated for range

Returns:

true if the year is a leap year

prolepticYear

```
public int prolepticYear(Era era,
                         int yearOfEra)
```

Description copied from interface: `Chronology`

Calculates the proleptic-year given the era and year-of-era.

This combines the era and year-of-era into the single proleptic-year field.

If the chronology makes active use of eras, such as `JapaneseChronology` then the year-of-era will be validated against the era. For other chronologies, validation is optional.

Specified by:

`prolepticYear` in interface `Chronology`

Parameters:

`era` - the era of the correct type for the chronology, not null

`yearOfEra` - the chronology year-of-era

Returns:

the proleptic-year

eraOf

```
public HijrahEra eraOf(int eraValue)
```

Creates the HijrahEra object from the numeric value. The Hijrah calendar system has only one era covering the proleptic years greater than zero. This method returns the singleton HijrahEra for the value 1.

Specified by:

eraOf in interface Chronology

Parameters:

eraValue - the era value

Returns:

the calendar system era, not null

Throws:

DateTimeException - if unable to create the era

eras

```
public List<Era> eras()
```

Description copied from interface: Chronology

Gets the list of eras for the chronology.

Most calendar systems have an era, within which the year has meaning. If the calendar system does not support the concept of eras, an empty list must be returned.

Specified by:

eras in interface Chronology

Returns:

the list of eras for the chronology, may be immutable, not null

range

```
public ValueRange range(ChronoField field)
```

Description copied from interface: Chronology

Gets the range of valid values for the specified field.

All fields can be expressed as a long integer. This method returns an object that describes the valid range for that value.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

This method will return a result whether or not the chronology supports the field.

Specified by:

range in interface Chronology

Parameters:

field - the field to get the range for, not null

Returns:

the range of valid values for the field, not null

resolveDate

```
public HijrahDate resolveDate(Map<TemporalField, Long> fieldValues,
                              ResolverStyle resolverStyle)
```

Description copied from class: `AbstractChronology`

Resolves parsed `ChronoField` values into a date during parsing.

Most `TemporalField` implementations are resolved using the resolve method on the field. By contrast, the `ChronoField` class defines fields that only have meaning relative to the chronology. As such, `ChronoField` date fields are resolved here in the context of a specific chronology.

`ChronoField` instances are resolved by this method, which may be overridden in subclasses.

- `EPOCH_DAY` - If present, this is converted to a date and all other date fields are then cross-checked against the date.
- `PROLEPTIC_MONTH` - If present, then it is split into the `YEAR` and `MONTH_OF_YEAR`. If the mode is strict or smart then the field is validated.
- `YEAR_OF_ERA` and `ERA` - If both are present, then they are combined to form a `YEAR`. In lenient mode, the `YEAR_OF_ERA` range is not validated, in smart and strict mode it is. The `ERA` is validated for range in all three modes. If only the `YEAR_OF_ERA` is present, and the mode is smart or lenient, then the last available era is assumed. In strict mode, no era is assumed and the `YEAR_OF_ERA` is left untouched. If only the `ERA` is present, then it is left untouched.
- `YEAR`, `MONTH_OF_YEAR` and `DAY_OF_MONTH` - If all three are present, then they are combined to form a date. In all three modes, the `YEAR` is validated. If the mode is smart or strict, then the month and day are validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the first month in the requested year, then adding the difference in months, then the difference in days. If the mode is smart, and the day-of-month is greater than the maximum for the year-month, then the day-of-month is adjusted to the last day-of-month. If the mode is strict, then the three fields must form a valid date.
- `YEAR` and `DAY_OF_YEAR` - If both are present, then they are combined to form a date. In all three modes, the `YEAR` is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the requested year, then adding the difference in days. If the mode is smart or strict, then the two fields must form a valid date.

- YEAR, MONTH_OF_YEAR, ALIGNED_WEEK_OF_MONTH and ALIGNED_DAY_OF_WEEK_IN_MONTH - If all four are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the first month in the requested year, then adding the difference in months, then the difference in weeks, then in days. If the mode is smart or strict, then the all four fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year and month, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the month.
- YEAR, MONTH_OF_YEAR, ALIGNED_WEEK_OF_MONTH and DAY_OF_WEEK - If all four are present, then they are combined to form a date. The approach is the same as described above for years, months and weeks in ALIGNED_DAY_OF_WEEK_IN_MONTH. The day-of-week is adjusted as the next or same matching day-of-week once the years, months and weeks have been handled.
- YEAR, ALIGNED_WEEK_OF_YEAR and ALIGNED_DAY_OF_WEEK_IN_YEAR - If all three are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the requested year, then adding the difference in weeks, then in days. If the mode is smart or strict, then the all three fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the year.
- YEAR, ALIGNED_WEEK_OF_YEAR and DAY_OF_WEEK - If all three are present, then they are combined to form a date. The approach is the same as described above for years and weeks in ALIGNED_DAY_OF_WEEK_IN_YEAR. The day-of-week is adjusted as the next or same matching day-of-week once the years and weeks have been handled.

The default implementation is suitable for most calendar systems. If ChronoField.YEAR_OF_ERA is found without an ChronoField.ERA then the last era in Chronology.eras() is used. The implementation assumes a 7 day week, that the first day-of-month has the value 1, that first day-of-year has the value 1, and that the first of the month and year always exists.

Specified by:

resolveDate in interface Chronology

Overrides:

resolveDate in class AbstractChronology

Parameters:

`fieldValues` - the map of fields to values, which can be updated, not null

`resolverStyle` - the requested type of resolve, not null

Returns:

the resolved date, null if insufficient information to create a date

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.